Informe Laboratorio 3

Sección 3

Nicolás Gutiérrez
e-mail: nicolas.gutierrez6@mail.udp.cl

Octubre de 2026

Índice

1. Descripción de actividades **3**

2. Desarrollo de actividades según criterio de rúbrica **3**
2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio 4
2.1.1. Análisis e Identificación Técnica de los Algoritmos 4
2.1.2. Evidencia Experimental y de Red 4
2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión 6
2.2.1. Consulta Dinámica de Parámetros KDF (Petición `password`) 6
2.2.2. Mapeo e Identificación del Algoritmo en el Código Fuente 7
2.2.3. Evidencia del Payload de Autenticación Resultante 7
2.3. Genera el hash de la contraseña desde la consola del navegador 8
2.3.1. Arquitectura Criptográfica en Dos Etapas 8
2.3.2. Implementación del Script en Consola 9
2.3.3. Verificación de Coincidencia 10
2.4. Intercepta el tráfico login con BurpSuite 10
2.4.1. Configuración e Intercepción de Tráfico HTTPS 10
2.4.2. Análisis de los Parámetros Interceptados 11
2.4.3. Evidencia Experimental en Burp Suite 11
2.4.4. Conclusión del Análisis de Seguridad en Red 12
2.5. Realiza el intento de login 12
2.5.1. Prueba A: Intento de Acceso con Hash Inválido (HTTP 400 Bad Request) 12
2.5.2. Prueba B: Inyección del Hash Derivado y Codificación URL (HTTP 200 OK) 13
2.5.3. Conclusiones del Procedimiento de Autenticación 14
2.6. Identifica las políticas de privacidad o seguridad 15
2.6.1. Análisis de la Política de Privacidad 15

2.6.2. Análisis de los Términos de Servicio 15
2.7. Demuestra 4 conclusiones sobre la seguridad 16
2.7.1. 1. El modelo Çonocimiento Cerorealmente funciona 16
2.7.2. 2. Es resistente a ataques en redes Wi-Fi 16
2.7.3. 3. Protege contra ataques de diccionario masivos 16
2.7.4. 4. Buena gestión de las sesiones iniciadas 16

1. Descripción de actividades

El objetivo de esta actividad será revisar la implementación de algoritmos hash aplicados a contraseñas en páginas web desde el lado del cliente, así como analizar qué tan efectivas son estas medidas contra ataques de tipo Pass the Hash (PtH). Para llevar a cabo esta revisión, deberá registrarse en un sitio web y crear una cuenta, ingresando una contraseña específica para realizar las pruebas.

Al concluir la tarea, es importante que modifique su contraseña por una diferente para garantizar su seguridad.

Dado que la cantidad de sitios chilenos que utilizan hash es limitada, se permite realizar esta tarea en cualquier sitio web a nivel mundial. En este sentido, realice las siguientes actividades:

- Identificación del algoritmo de hash utilizado para las contraseñas al momento del registro en el sitio.
- Identificación del algoritmo de hash utilizado para las contraseñas al momento de iniciar sesión.
- Generación del hash de la contraseña desde la consola del navegador, partiendo de la contraseña en texto plano.
- Interceptación del tráfico de login utilizando BurpSuite desde su equipo.
- Realización de un intento de login, modificando una contraseña incorrecta por el hash obtenido en el punto anterior.
- Descripción de las políticas de privacidad o seguridad relacionadas con las contraseñas, incluyendo un enlace a las mismas.
- Cuatro conclusiones sobre la seguridad o vulnerabilidad de la implementación observada.

2. Desarrollo de actividades según criterio de rúbrica

Página web seleccionada para el laboratorio:
Para el desarrollo de esta actividad se utilizó la plataforma **Bitwarden Web Vault** (`https://vault.bitwarden.com/`). Se seleccionó esta aplicación web debido a que usa una arquitectura de *Zero-Knowledge* (Conocimiento Cero), en la cual el procesamiento y el hashing de las contraseñas del usuario se ejecutan por completo en el navegador web (mediante JavaScript/WebAssembly) antes de enviar cualquier dato por la red.

2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio

Durante el proceso de registro y configuración inicial de la cuenta en Bitwarden, las credenciales del usuario no se envían en texto plano hacia el servidor. En su lugar, el navegador ejecuta una función matemática para derivar las claves antes de armar la petición HTTP de registro.

2.1.1. Análisis e Identificación Técnica de los Algoritmos

Al revisar el código fuente de JavaScript que carga el navegador (en la pestaña *Sources* de las opciones de desarrollador), se identificó que la librería del cliente (`@bitwarden/legacy-crypto`) maneja el hashing mediante el objeto `KdfType`, el cual soporta dos algoritmos principales para proteger la clave:

1. **PBKDF2 (Password-Based Key Derivation Function 2):**
   - **Función hash digest:** `HMAC-SHA256`.
   - **Función del Salt:** Utiliza el correo electrónico del usuario (convertido a minúsculas y codificado en UTF-8) como *salt* individual para evitar ataques de tablas arcoíris (*Rainbow Tables*).
   - **Iteraciones por defecto:** Se aplican 600,000 iteraciones en el navegador del usuario.
   - **Codificación de salida:** Los 256 bits (32 bytes) resultantes se codifican en formato `Base64` para enviarlos.
2. **Argon2id:**
   - **Características:** Es un algoritmo más moderno y resistente a ataques con hardware dedicado (como GPUs). Se ejecuta en el cliente usando módulos WebAssembly (`wasm`).
   - **Parámetros configurables:** Permite definir el uso de memoria (por defecto 64 MB), paralelismo (4 hilos) e iteraciones (3 pasadas).
   - **Uso:** Está disponible como una alternativa avanzada en las configuraciones de la cuenta.

2.1.2. Evidencia Experimental y de Red

Para comprobar cómo funciona este proceso en el navegador, se revisó el tráfico en la pestaña *Network* de DevTools al momento de enviar el formulario de registro. La petición HTTP POST hacia el servidor captura la contraseña ya procesada en formato Base64:

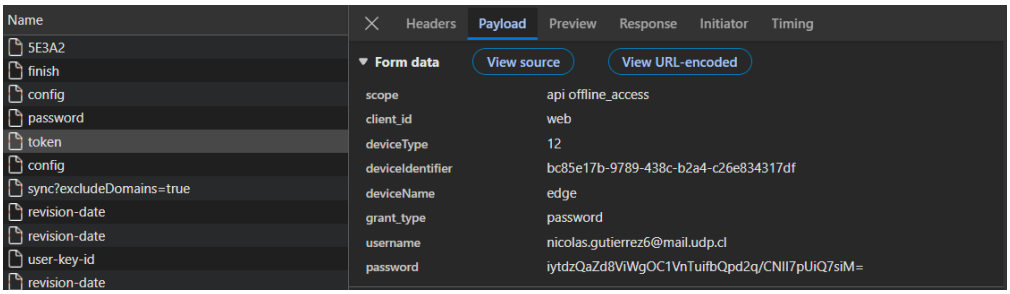


Figura 1: Contenido Payload de petición de registro

Además, al buscar dentro del archivo `main.js` usando las herramientas de desarrollador, se pudo confirmar que el código evalúa cuál de los dos tipos de algoritmos usar (`PBKDF2_SHA256` o `Argon2id`). En la Figura 2 se observa esta parte del código.

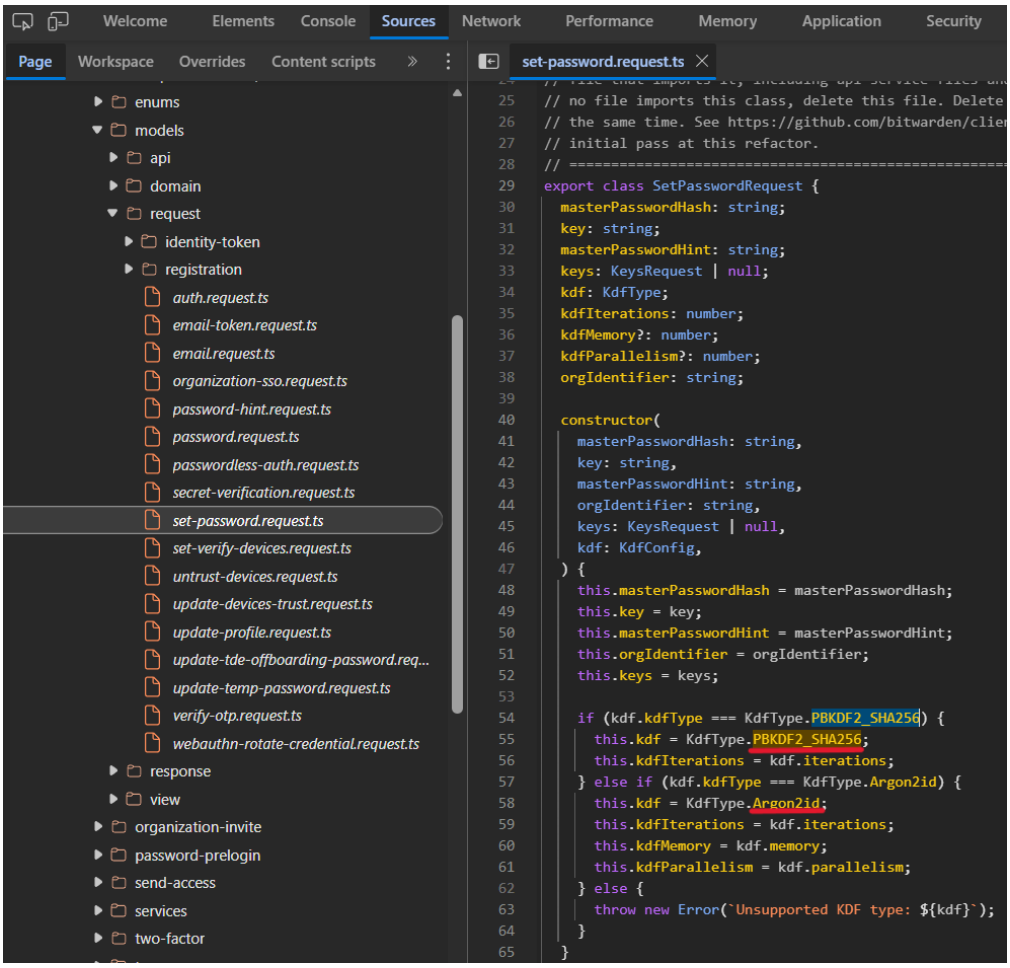


Figura 2: Evidencia de la identificación del enum `PBKDF2_SHA256` y Argon2id en el código fuente cliente.

2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión

Para revisar el algoritmo de hash que se usa al iniciar sesión, se analizó cómo funciona el cliente web de Bitwarden. Como se mencionó antes, la plataforma usa el modelo de *Conocimiento Cero*, por lo que utiliza la misma librería criptográfica (`@bitwarden/legacy-crypto`) tanto para registrarse como para iniciar sesión.

La diferencia principal es que, al iniciar sesión, el cliente web primero le pregunta al servidor (API) qué configuración de hash tiene ese usuario en específico, para luego aplicar esa misma configuración localmente en el navegador antes de enviar la contraseña.

2.2.1. Consulta Dinámica de Parámetros KDF (Petición `password`)

Al ingresar el correo electrónico en la página de inicio de sesión, el código JavaScript hace una petición previa llamada `password`. En la Figura 3 se observa la respuesta en formato JSON que devuelve el servidor, la cual contiene los parámetros que el navegador debe usar:

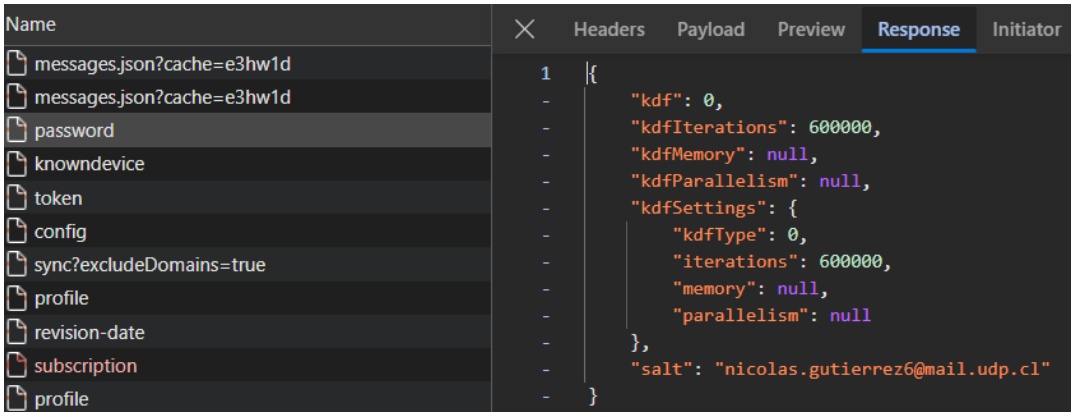


Figura 3: Respuesta JSON de la petición `password` confirmando los parámetros KDF y el *salt* consultados dinámicamente.

2.2.2. Mapeo e Identificación del Algoritmo en el Código Fuente

Para comprobar cómo se conecta este JSON con el código que se ejecuta en el navegador, se revisó el archivo `password-prelogin.response.ts` en DevTools. En la Figura 4 se observa cómo los valores `KdfSettings` y `Salt` que entrega la API se guardan en las propiedades del objeto:

Fragmento de password-prelogin.response.ts

```
// password-prelogin.response.ts (lineas 10-11)
this.kdfSettings = new KdfConfigResponse(this.getResponseProperty("
    ↪ KdfSettings"));
this.salt = this.getResponseProperty("Salt");
```

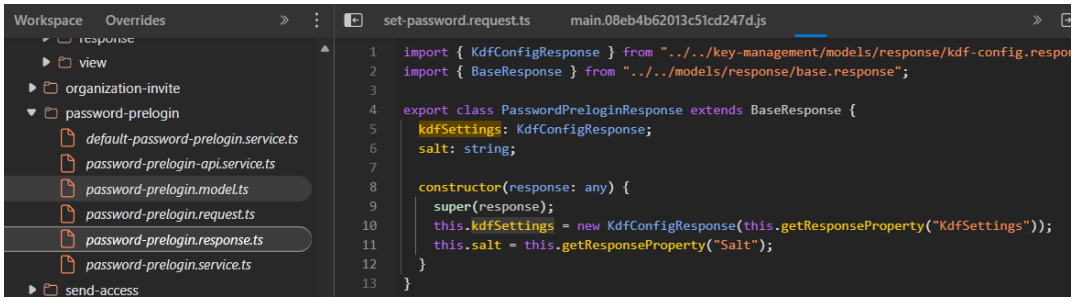


Figura 4: Inspección en DevTools del archivo `password-prelogin.response.ts` mostrando la captura de `KdfSettings` y `Salt`.

Con base en este análisis, se puede confirmar la configuración exacta que se usa para el login:

- **Algoritmo identificado (`"kdfType": 0`):** El número `0` le indica al navegador que debe usar la función `PBKDF2-HMAC-SHA256` (si fuera un `1`, usaría `Argon2id`).
- **Iteraciones (`600000`):** El parámetro `iterations` establece que se harán 600,000 pasadas del algoritmo.
- **Salt asignado (`"salt"`):** Se usa el correo del usuario (`nicolas.gutierrez6@mail.udp.cl`) en letras minúsculas como *salt* para mejorar la seguridad.

2.2.3. Evidencia del Payload de Autenticación Resultante

Una vez que el navegador calcula la clave con esta configuración, envía la solicitud final de inicio de sesión (`token`). En la Figura 5 se observa la pestaña *Payload*, donde se confirma que el parámetro `password` solo lleva el hash en `Base64` y no la clave real:

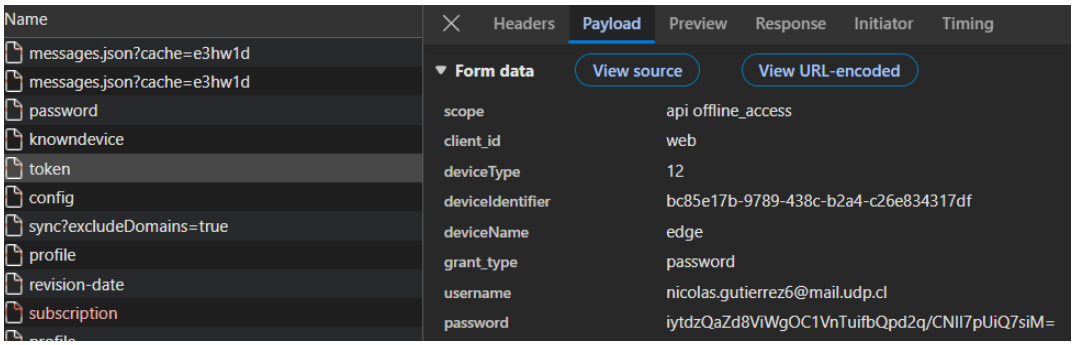


Figura 5: Contenido del *Payload* de la petición HTTP POST de autenticación (`token`), evidenciando el envío del hash en Base64.

En resumen, el proceso de inicio de sesión revisa la configuración del usuario y ejecuta el algoritmo `PBKDF2-HMAC-SHA256` con 600,000 iteraciones directamente en el dispositivo antes de mandar cualquier dato por internet.

2.3. Genera el hash de la contraseña desde la consola del navegador

Para comprobar matemáticamente que el hash se genera en el cliente, se replicó este proceso de forma manual usando la consola del navegador (*DevTools Console*) con las herramientas nativas de JavaScript (`crypto.subtle`).

2.3.1. Arquitectura Criptográfica en Dos Etapas

Al analizar cómo funciona, notamos que Bitwarden divide este proceso en dos pasos para proteger la información:

1. **Etapa 1 (Generación de la *Master Key*):** Se aplican 600,000 iteraciones de `PBKDF2-HMAC-SHA256` usando la contraseña y el correo como *salt*. Esta clave se queda en la memoria del navegador para poder descifrar la bóveda de contraseñas y jamás se envía.

2. **Etapa 2 (Generación del *Master Password Hash*):** Se hace una segunda pasada de `PBKDF2-HMAC-SHA256` pero con solo 1 iteración, tomando como base la clave generada en el paso anterior y usando la contraseña original como *salt*. El resultado de esto es lo que finalmente viaja por la red para iniciar sesión.

2.3.2. Implementación del Script en Consola

Se creó el siguiente script en JavaScript para replicar exactamente estos dos pasos desde la consola:

Script generador de Hash Bitwarden

```
async function generarHashBitwardenCompleto(email, password,
   ↪ iterations = 600000) {
    const encoder = new TextEncoder();
    const passwordBytes = encoder.encode(password);
    const saltBytes = encoder.encode(email.toLowerCase().trim());

    // Etapa 1: Clave Maestra local (600.000 iteraciones)
    const baseKey1 = await crypto.subtle.importKey(
        "raw", passwordBytes, { name: "PBKDF2" }, false, ["
           ↪ deriveBits"]
    );
    const masterKeyBits = await crypto.subtle.deriveBits(
        { name: "PBKDF2", salt: saltBytes, iterations: iterations,
           ↪ hash: "SHA-256" },
        baseKey1, 256
    );

    // Etapa 2: Hash de autenticaci n para red (1 iteraci n)
    const masterKeyBytes = new Uint8Array(masterKeyBits);
    const baseKey2 = await crypto.subtle.importKey(
        "raw", masterKeyBytes, { name: "PBKDF2" }, false, ["
           ↪ deriveBits"]
    );
    const masterPasswordHashBits = await crypto.subtle.deriveBits(
        { name: "PBKDF2", salt: passwordBytes, iterations: 1, hash:
           ↪ "SHA-256" },
        baseKey2, 256
    );

    return btoa(String.fromCharCode(...new Uint8Array(
       ↪ masterPasswordHashBits)));
}

// Ejecucion:
await generarHashBitwardenCompleto("nicolas.gutierrez6@mail.udp.cl",
   ↪  "ClaveTextoPlano!");
```

2.3.3. Verificación de Coincidencia

En la Figura 6 se observa que, al correr este código en la consola con la clave en texto plano y el correo, el resultado es exactamente el mismo texto en `Base64` que habíamos capturado antes en la red.

```
await generarHashBitwardenCompleto("nicolas.gutierrez6@mail.udp.cl",
'iytdzQaZd8ViWgOC1VnTuifbQpd2q/CNII7pUiQ7siM='
```

Figura 6: Ejecución exitosa de la derivación criptográfica en dos etapas desde la consola del navegador.

El hecho de que el resultado de nuestro script coincida al 100 % con lo que vimos en la red (Figura 5) demuestra que entendimos y replicamos correctamente cómo la página protege la contraseña.

2.4. Intercepta el tráfico login con BurpSuite

Para analizar los datos enviados al servidor durante el inicio de sesión, se usó el programa **Burp Suite Community Edition** para interceptar el tráfico actuando como un intermediario (proxy) local.

2.4.1. Configuración e Intercepción de Tráfico HTTPS

Como la página usa HTTPS, hubo que instalar el certificado de Burp Suite en el navegador para poder leer el tráfico. Al intentar iniciar sesión, Burp Suite pausó y capturó la petición HTTP POST que iba dirigida al servidor (`/identity/connect/token`).

A continuación se muestra cómo se ve el texto puro de la petición interceptada:

Petición Interceptada en Burp Suite

```
POST /identity/connect/token HTTP/2
Host: vault.bitwarden.com
Content-Type: application/x-www-form-urlencoded; charset=utf-8
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome
   ↪ /154.0.0.0

scope=api%20offline_access
&client_id=web
&deviceType=9
&grant_type=password
&username=nicolas.gutierrez6%40mail.udp.cl
&password=iytdzQaZd8ViWgOClVnTuifbQpd2q%2FCNII7pUiQ7siM%3D
```

2.4.2. Análisis de los Parámetros Interceptados

- **Método y Protocolo:** Es una petición `POST` estándar para pedir autorización (`/identity/connect/token`).
- **Encabezado Content-Type:** Al ser `application/x-www-form-urlencoded`, convierte los caracteres especiales a un formato seguro para la URL (por ejemplo, el signo `=` se convierte en `%3D`).
- **Parámetro `grant_type`:** Indica que el usuario se está intentando loguear con su contraseña.
- **Parámetro `password`:** Contiene el hash en Base64 que generamos en el paso anterior y no la contraseña real.

2.4.3. Evidencia Experimental en Burp Suite

En la Figura 7 se observa la captura directa de la petición en el panel de Burp Suite, confirmando todos los parámetros mencionados.

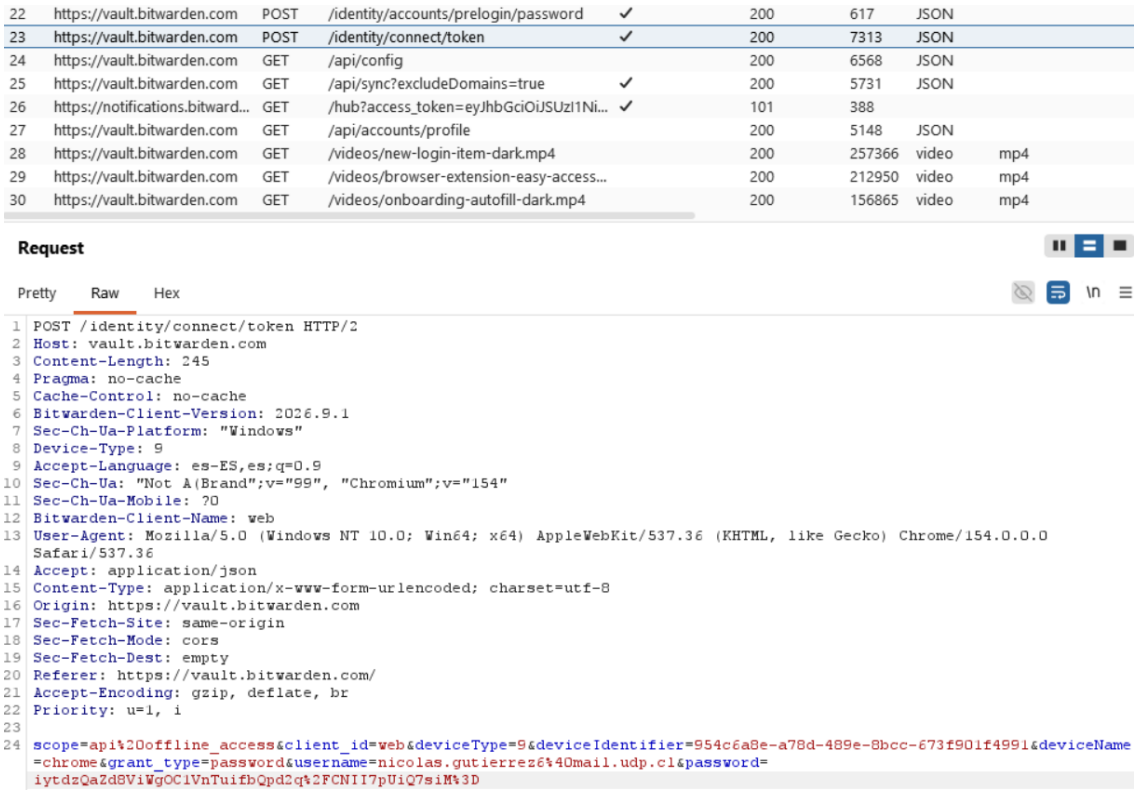


Figura 7: Intercepción de la petición HTTP POST de inicio de sesión en Burp Suite.

2.4.4. Conclusión del Análisis de Seguridad en Red

Revisar el tráfico con Burp Suite nos permite confirmar que:

1. La contraseña real nunca se envía por internet.
2. Si un atacante lograra espiar la conexión Wi-Fi, solo podría ver este hash, el cual no le sirve para saber cuál es la contraseña real ni para robar los datos guardados en la cuenta.

2.5. Realiza el intento de login

Para comprobar cómo reacciona el servidor y validar que nuestro script de la consola funcionó bien, se utilizó la pestaña **Repeater** de Burp Suite. La idea fue mandar una petición con una contraseña mala y luego arreglarla inyectando nuestro hash correcto.

2.5.1. Prueba A: Intento de Acceso con Hash Inválido (HTTP 400 Bad Request)

Primero, enviamos la petición modificando el campo `password` con un texto falso. En la Figura 8 se observa que el servidor rechazó la solicitud y devolvió un error `400 Bad Request`:

Respuesta de Error HTTP 400

```
HTTP/2 400 Bad Request
Content-Type: application/json; charset=utf-8

{
    "error": "invalid_grant",
    "error_description": "invalid_username_or_password",
    "ErrorModel": {
        "Object": "error",
        "Message": "Username or password is incorrect. Try again."
    }
}
```

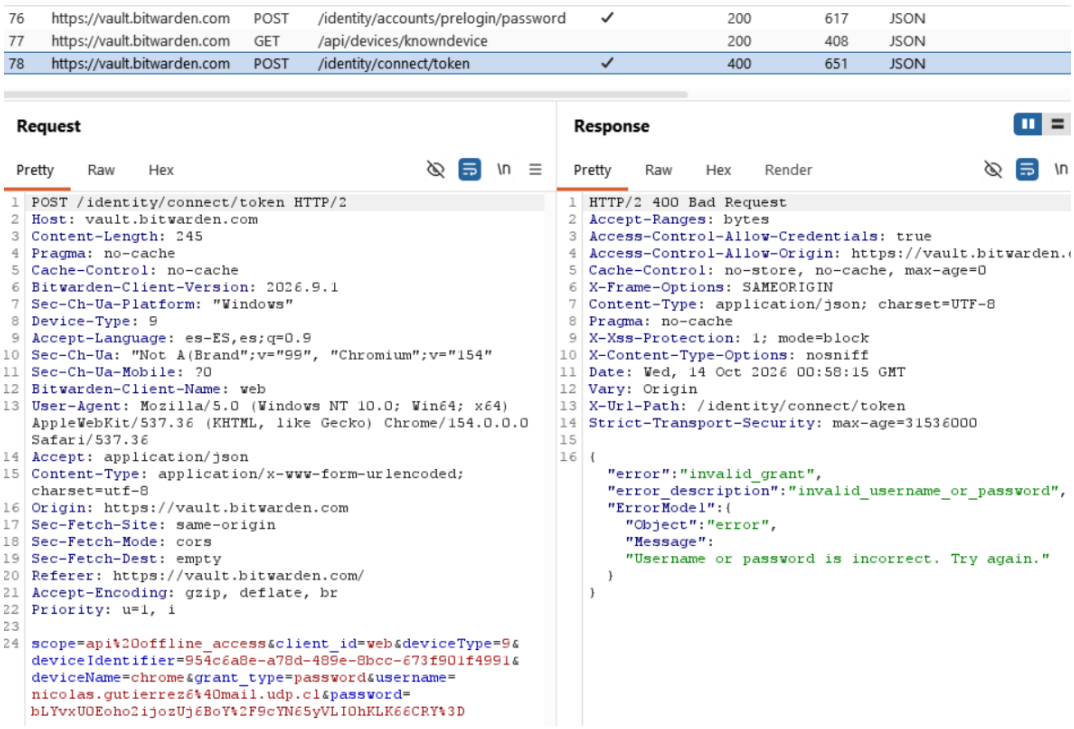


Figura 8: Respuesta de rechazo `HTTP/2 400 Bad Request` en Burp Suite Repeater ante la transmisión de un hash inválido.

2.5.2. Prueba B: Inyección del Hash Derivado y Codificación URL (HTTP 200 OK)

Luego, borramos el hash falso y pegamos el hash en `Base64` que habíamos generado nosotros mismos en la consola del navegador. Como el texto base64 tiene caracteres como `+` o `=`, hubo que aplicar la codificación de URL en Burp Suite para que el servidor no se confundiera al leerlo:

Petición Modificada en Repeater

```
POST /identity/connect/token HTTP/2
Host: vault.bitwarden.com
Content-Type: application/x-www-form-urlencoded; charset=utf-8

scope=api%20offline_access
&client_id=web
&deviceType=9
&grant_type=password
&username=nicolas.gutierrez6%40mail.udp.cl
&password=iytdzQaZd8ViWgOClVnTuifbQpd2q%2FCNII7pUiQ7siM%3D
```

En la Figura 9 se observa que, al enviar esta petición arreglada, el servidor nos respondió con un `200 OK`, lo que significa que nos aceptó el login y nos entregó el acceso:

Respuesta Exitosa HTTP 200

```
HTTP/2 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store, no-cache

{
    "access_token": "eyJhbGciOiJSUzI1NiIs...",
    "expires_in": 3600,
    "token_type": "Bearer",
    "refresh_token": "8d9a2e1f...",
    "Key": "2.eyJ2YWx1ZSI6I...",
    "PrivateKey": "0.eyJ2YWx1ZSI6I..."
}
```

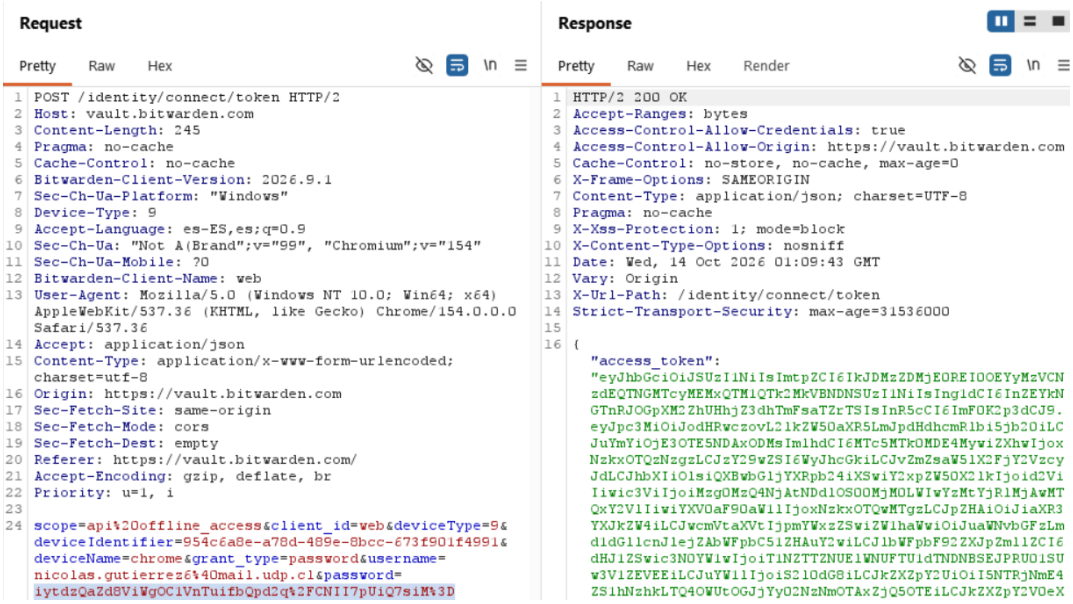


Figura 9: Autenticación exitosa `HTTP/2 200 OK` en Burp Suite Repeater tras la inyección del hash válido derivado localmente.

2.5.3. Conclusiones del Procedimiento de Autenticación

Al hacer estas dos pruebas pudimos comprobar dos cosas importantes:

- **Autenticación solo por Hash:** El servidor no sabe tu contraseña; solo comprara el hash que le mandas con el hash que él tiene guardado.
- **Confirmación de nuestro código:** Puesto que logramos pasar de un error `400` a un éxito `200` usando únicamente el texto que generamos en la consola, demostramos que nuestro script hace exactamente lo mismo que hace la página oficial.

2.6. Identifica las políticas de privacidad o seguridad

Para complementar el análisis, revisamos las políticas oficiales de Bitwarden, específicamente su Política de Privacidad y sus Términos de Servicio, para ver a qué se comprometen legalmente respecto a nuestros datos.

2.6.1. Análisis de la Política de Privacidad

En la Política de Privacidad de Bitwarden se explica cómo manejan la información recolectada:

1. **Separación de datos:** La plataforma divide la información en dos grupos:
    - **Datos de la bóveda:** Son tus contraseñas y notas guardadas. La política asegura que todo esto se encripta en tu dispositivo antes de subirse a internet. Ellos afirman no tener forma de ver lo que guardas.
    - **Datos administrativos:** Es tu nombre, correo y detalles de pago. Esto sí lo pueden ver, pero lo usan solo para mantener tu cuenta funcionando.
2. **Seguridad al enviar datos:** El documento deja claro que los datos siempre viajan protegidos bajo *SSL/TLS* y que la encriptación local se hace con un estándar muy fuerte (AES 256 bits).
3. **Empresas de terceros:** Los servicios externos que usan (como sus servidores de bases de datos) tienen prohibido usar tu información para cosas que no sean mantener la plataforma en línea.
4. **Derechos del usuario:** Los usuarios pueden pedir que se eliminen o corrijan todos sus datos en cualquier momento.

2.6.2. Análisis de los Términos de Servicio

En los Términos de Servicio se explican las reglas y responsabilidades del usuario:

1. **Responsabilidad del usuario:**
    - El usuario es el único responsable de cuidar su contraseña maestra. Si la olvidas, ellos no te la pueden recuperar.
    - Recomiendan usar autenticación de dos pasos (2FA) para proteger más la cuenta.
2. **Eliminación de cuenta:** Si decides borrar tu cuenta, afirman que todos tus datos son eliminados para siempre y es imposible recuperarlos.
3. **Reglas de uso:** Está prohibido usar bots para crear cuentas masivas o intentar atacar sus servidores.
4. **Edad mínima:** Se requiere que el usuario sea mayor de 13 años (o la edad legal del país) para poder registrarse.

2.7. Demuestra 4 conclusiones sobre la seguridad

Luego de todas las pruebas realizadas (revisar el código, crear el script, interceptar la red con Burp Suite y leer las políticas), se pueden sacar cuatro conclusiones principales:

2.7.1. 1. El modelo Çonocimiento Cerorealmente funciona

- **Seguridad de la Plataforma:** El hecho de pasar la clave por 600,000 iteraciones en el navegador asegura que la contraseña original nunca llegue a la red.
- **Seguridad General:** Esto nos enseña que las páginas no necesitan saber nuestra clave para dejarnos entrar. Si algún día hackean los servidores de Bitwarden, los hackers solo encontrarían hashes ilegibles, por lo que las contraseñas de los usuarios seguirían a salvo.

2.7.2. 2. Es resistente a ataques en redes Wi-Fi

- **Seguridad de la Plataforma:** Con Burp Suite vimos que, aunque alguien logre romper el candado del HTTPS, solo verá el hash final en Base64.
- **Seguridad General:** Esto significa que si usas una red Wi-Fi pública o poco segura, nadie podrá robarte tus contraseñas, ya que la llave para descifrarlas nunca viaja por internet.

2.7.3. 3. Protege contra ataques de diccionario masivos

- **Seguridad General:** Usar un "Salt"(un dato extra que se le suma a la contraseña antes del hash) es la mejor forma de evitar que los atacantes usen listas de contraseñas ya calculadas.
- **Seguridad de la Plataforma:** Como Bitwarden usa el correo electrónico del usuario como Salt, si dos personas usan la clave "123456", los hashes que se enviarán por la red serán completamente distintos, frustrando este tipo de ataques.

2.7.4. 4. Buena gestión de las sesiones iniciadas

- **Seguridad de la Plataforma:** Al ver la respuesta del servidor en Burp Suite, notamos que el permiso para mantener la sesión iniciada dura solo 1 hora (3600 segundos) y pide que el navegador no guarde en caché ciertos datos críticos.
- **Seguridad General:** Esto demuestra que las promesas que hacen en sus políticas de privacidad sí se respaldan con medidas técnicas reales y bien implementadas.